

Assignment - 4 C++

Section - A

Q.1 Differentiate between constant class member and static class member with suitable examples.

Ans

Feature	Constant member	Static Member
Definition	Value cannot be changed after initialization	Shared among all objects.
Memory	Separate for each object	Single copy for class.
Keywords	const	static
Initialization	Usually in constructor	Outside class.

E.g :- class Test {

const int x; // Constant member

static int y; // Static member

public:

Test(int a): x(a) {}

void show() {

cout << "x =" << x << ", y =" << y << endl;

}

};

Q.2 Write a C++ program to demonstrate single inheritance.

Ans

#include <iostream>

using namespace std;

class Base {

```
public:  
    void display () {  
        cout << "This is Base class\n";  
    }  
};
```

```
class Derived : public Base {  
public:  
    void show () {  
        cout << "This is Derived class\n";  
    }  
};
```

```
int main () {  
    Derived obj;  
    obj.display ();  
    obj.show ();  
}
```

Q.3 Define virtual function and state its importance.

Ans A virtual function is a member function declared with virtual keyword and is overridden in derived class.

Importance :-

- Supports runtime polymorphism
- Enable dynamic binding
- Ensures correct function call using base pointer.

Q.4 write a program to show hybrid inheritance.

```
#include <iostream>
using namespace std;

class A {
public:
    void showA() { cout << "class A\n"; }
};

class B : public A {
};

class C {
public:
    void showC() { cout << "class C\n"; }
};

class D : public B, public C {
};

int main() {
    D obj;
    obj.showA();
    obj.showC();
}
```

Q.5 Explain base class and derived class with example.

Ans

- Base class : Parent class.
- Derived class : Child class inheriting properties.

E.g :- class Animal {

public :

void eat() { cout << "Eating\n"; }

};

class Dog : public Animal {

public :

void bark() { cout << "Barking\n"; }

};

Section - B

Q.1 Explain different types of inheritance in C++ with diagram and examples.

Ans

1. Single → One base → one derived
2. Multiple → Multiple base → one derived
3. Multilevel → Chain inheritance
4. Hierarchical → One base → multiple derived
5. Hybrid → Combination

E.g. (Multilevel) :-

class A {};

class B : public A {};

class C : public B {};

Q.2 Describe constructor and destructor calls in inheritance hierarchy with example.

- Ans .
- Constructor are called top-down.
 - Destructor are called bottom-up.

E.g :- # include <iostream>
using namespace std;

class Base {
public:

Base () { cout << "Base Constructor \n"; }

~ Base () { cout << "Base Destructor \n"; }

};

class Derived : public Base {
public:

Derived () { cout << "Derived Constructor \n"; }

~ Derived () { cout << "Derived Destructor \n"; }

};

int main () {
Derived obj;
}

Q.3 Explain multiple inheritance and discuss ambiguity problem with example.

Ans Multiple inheritance is a type of inheritance in which a derived class inherits properties and behaviours from more than one base class.

Problem : Same function in multiple base classes

E.g :- # include <iostream>
using namespace std;

```
class A {  
public :  
    void show () { cout << "class A\n"; }  
};
```

```
class B {  
public :  
    void show () { cout << "class B\n"; }  
};
```

```
class C : public A, public B {  
};
```

```
int main () {  
    C obj;  
    obj.A::show (); // resolve ambiguity  
    obj.B::show ();  
}
```

Section - C

Q.1 Explain virtual base class with suitable C++ program and diagram.

Ans A virtual base class is used in C++ to avoid multiple copies of a base class when using multiple inheritance.

E.g :- #include <iostream>
using namespace std;

```
class A {
```

```
public :  
    int x;  
};  
  
class B : virtual public A {};  
class C : virtual public A {};  
  
class D : public B, public C {};  
  
int main () {  
    D obj;  
    obj.x = 10; // only one copy exists  
    cout << obj.x;  
}
```

Q.2 Discuss virtual function and runtime polymorphism with detailed example.

Ans

- Virtual function : A virtual function is a member function of a class that is declared using the virtual keyword and is overridden in a derived class.

It is used to achieve runtime polymorphism.

- Runtime polymorphism : Runtime polymorphism is the ability to decide which function to call at runtime based on the object type, not the pointer type.

E.g:- #include <iostream>
using namespace std;

```
class Base {  
public:  
    virtual void show () {  
        cout << "This is Base class\n";  
    }  
};
```

```
class Derived : public Base {  
public:  
    void show () {  
        cout << "This is Derived class\n";  
    }  
};
```

```
int main () {  
    Base* ptr;  
    Derived d;  
  
    ptr = &d;  
    ptr -> show (); // calls Derived version (runtime decision)  
}
```

Q.3 Explain static and constant member functions in detail with examples.

Ans

- Static member function : A static member function is a function that belongs to the class rather than any object. It can access only static data member of the class.

Key features :

- Declared using static keyword
- Called using class name.
- Cannot access non-static member directly.
- No this pointer.

E.g :- class Test {
static int count; // static data member

public:

Test () {

count ++;

}

static void showCount () { // static member function
cout << "Count = " << count << endl;

}

};

int Test :: count = 0;

int main () {

Test t1, t2, t3;

Test :: showCount (); // called using class name
}

- Constant member function : A constant member function that does not modify any data members of the class. It is declared using const keyword at the end of the function.

Key Features : • Cannot modify object data.
• Can call only other const functions.
• Used with constant objects.

E.g :- class Test {
 int x;

public :

 Test (int a) {
 x = a;
 }

 void show () const { // constant function
 cout << "value of x = " << x << endl;
 }

};

int main () {

 Test obj (10);
 obj.show ();

}

~~Flow~~
9/10/20